

Deep Learning 3

Recurrent networks

Machine learning

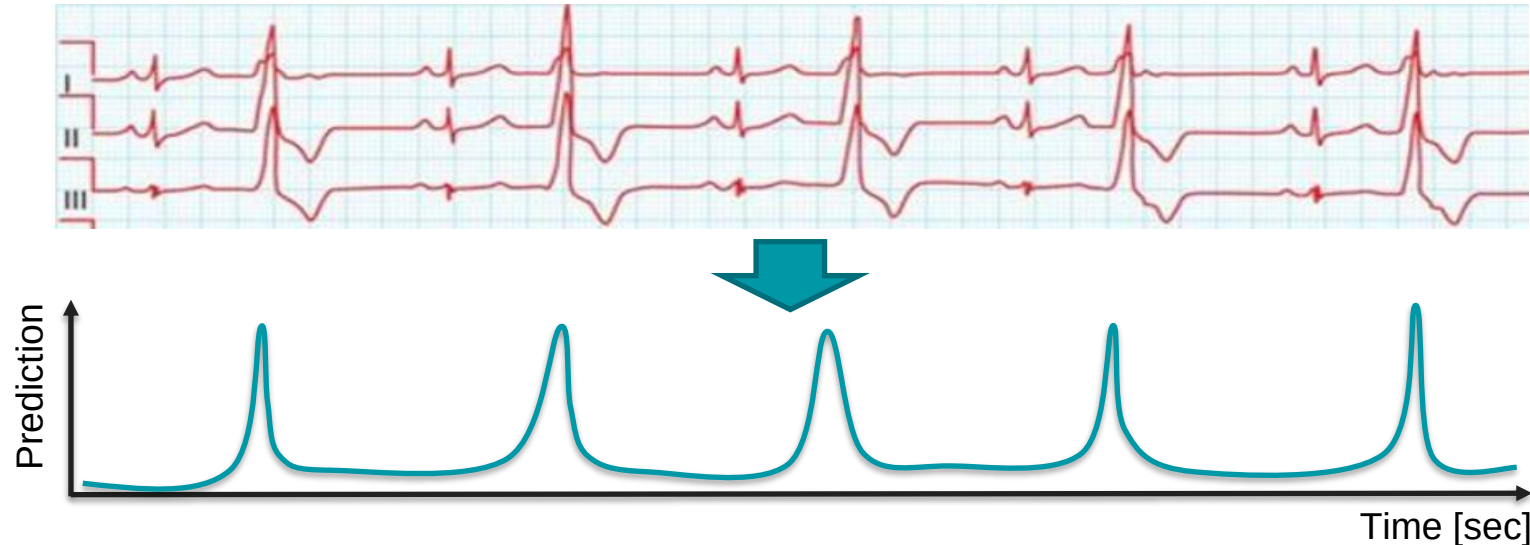
Roman Jakubicek

Department of Biomedical Engineering

Recurrent networks

Motivation

- Analysis (prediction/classification) of data through any dimension (time, space, ...), e.g. prediction of pathologies from a multi-channel electrocardiogram
- This dimension can be e.g. a sequence of words in a sentence (text analysis or generation) or gene sequence
- Moreover, there is a problem that the sequences may be a different length (e.g. ECG signals) => the standard networks have fixed number of input neurons



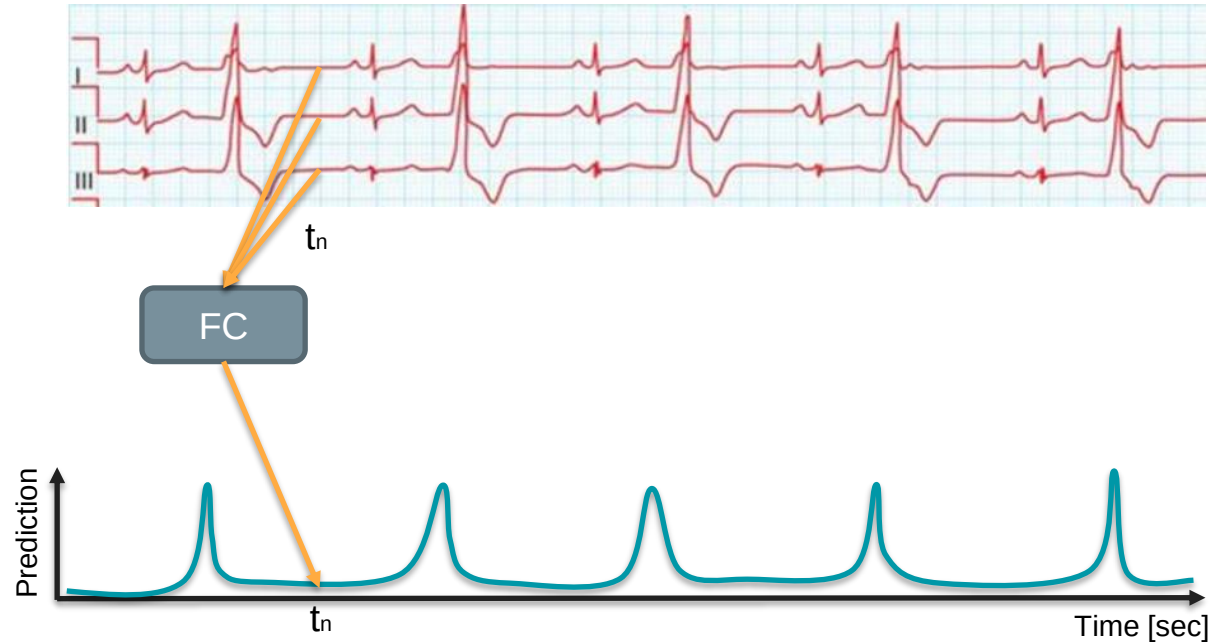
Solution

- Prediction of premature ventricular contractions in each point of dimension (in this case it is time). We can learn a single shallow or deep FC network and predicate “probability” for each time point of the signals.



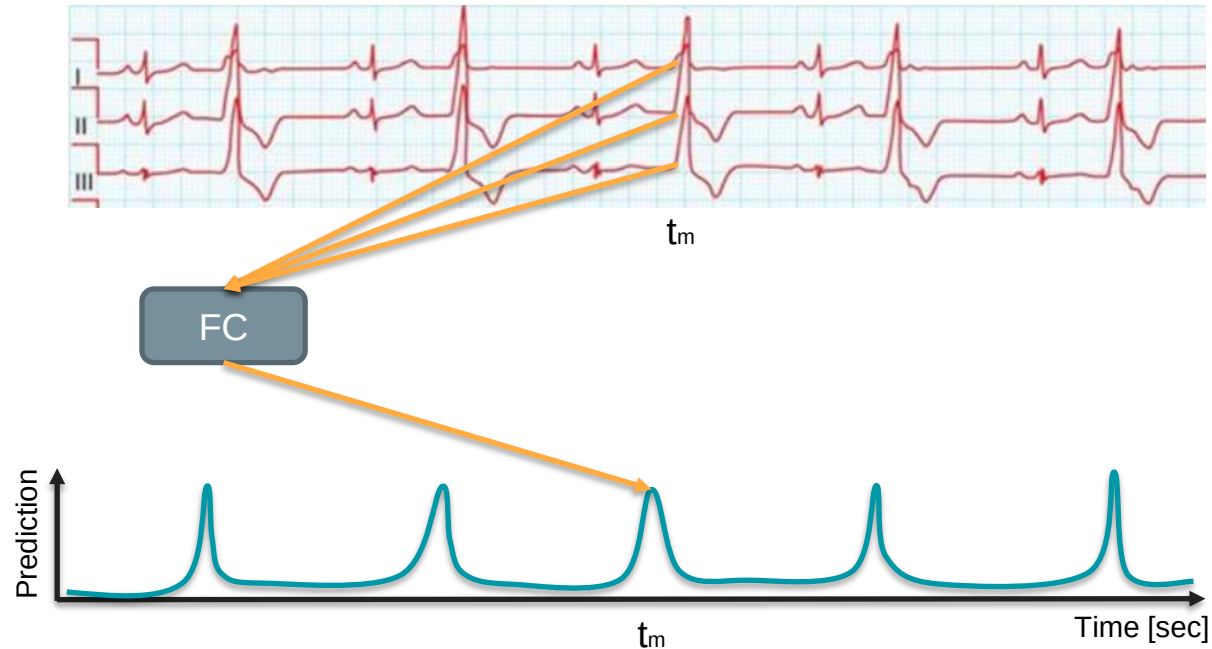
Solution

- Prediction of premature ventricular contractions in each point of dimension (in this case it is time). We can learn a single shallow or deep FC network and predicate “probability” for each time point of the signals.



Solution

- Prediction of premature ventricular contractions in each point of dimension (in this case it is time). We can learn a single shallow or deep FC network and predicate “probability” for each time point of the signals.



Solution

- Prediction of premature ventricular contractions in each point of dimension (in this case it is time). We can learn a single shallow or deep FC network and predicate “probability” for each time point of the signals.
- **But...** we consider **no neighbor time points**, i.e. the prediction is based on only the actual value of signals

Solution

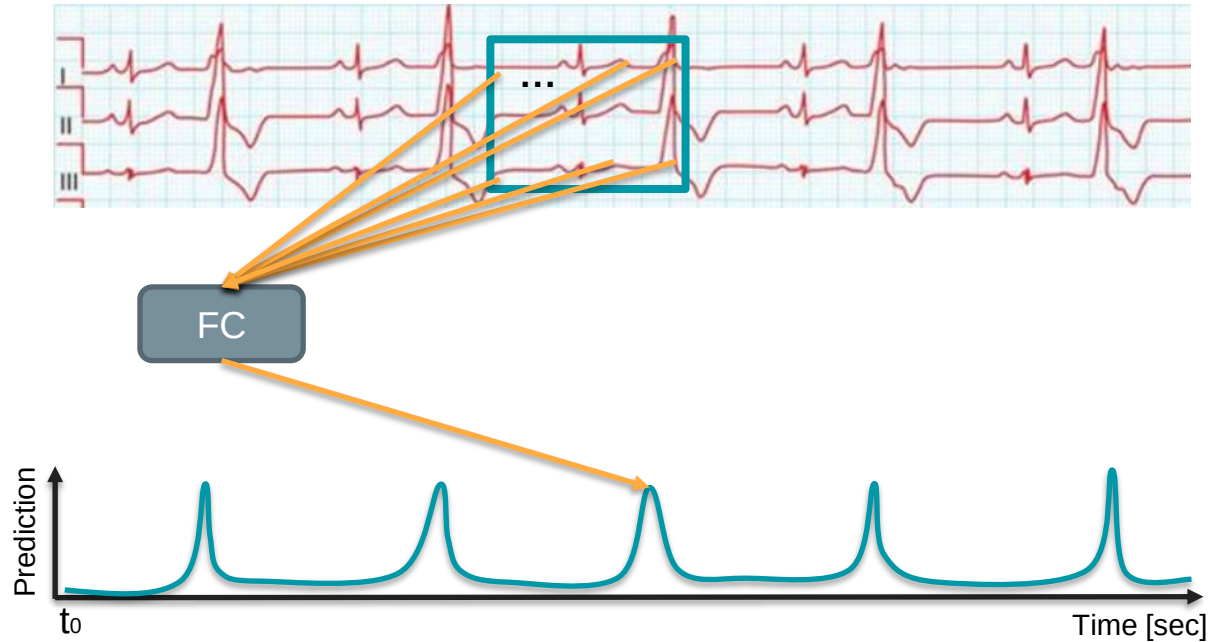
- Extension of number of an input value into the input layer (=> larger number of neurons)
- It is analogous to “sliding window”
- We must define the size of “window”

Usually, we want long time window

Larger window => higher computational complexity

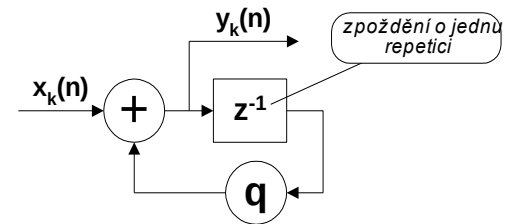
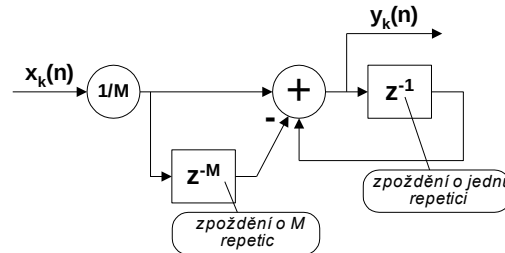
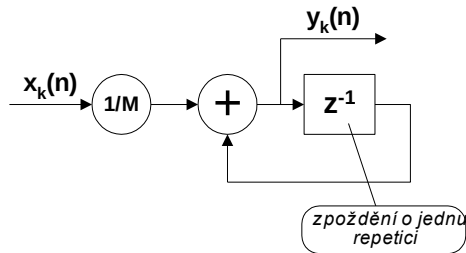
Solution

- Application of the “sliding window”
- Divide the signals into segments (N – length of window) and create a training database
- Number of neurons: $N \times d$ (d ... number of features/signals)



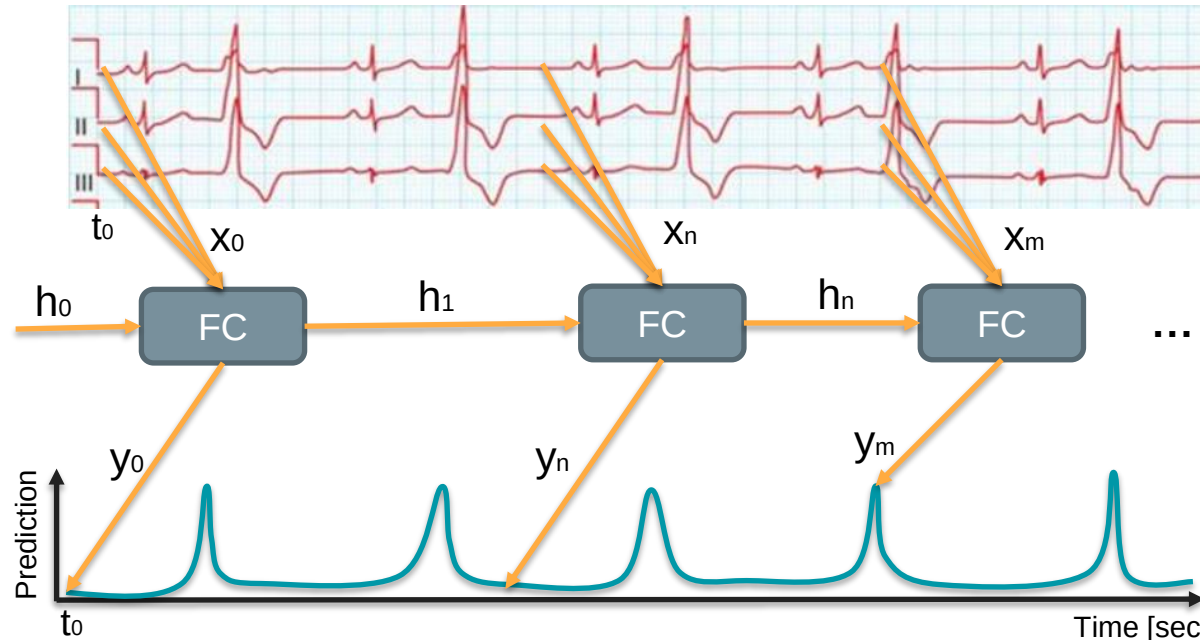
Cumulation Note

- We can process the sequence as piecewise problem (cumulative) => there is a risk of saturation/ exploding => we need to introduce a **forgetting** system
- in the signal processing we know three systems of the cumulation – with fixed window, sliding window or exponential forgetting
- in the neural networks, we can use a **controlled forgetting** via fully connected layers



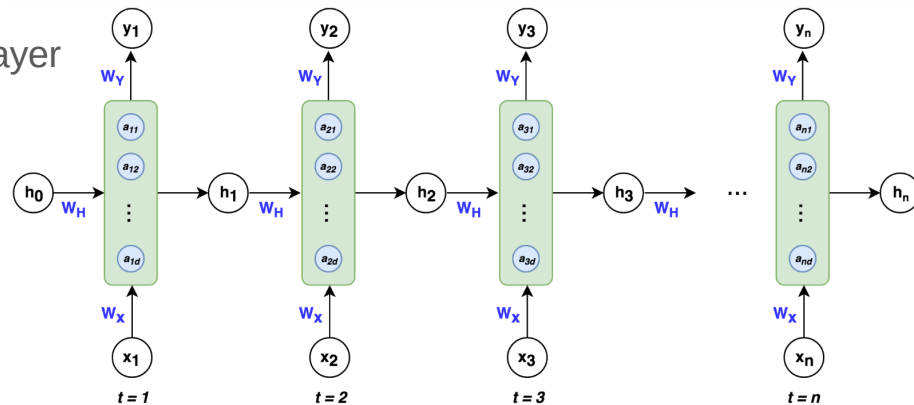
Recurrent Networks

- this network introduces new input ($\mathbf{h}$) vector, which is output from network in previous time point
- FC has thus two input vectors ($\mathbf{h}_{k-1}$ and $\mathbf{x}_{k-1}$) and two outputs ($\mathbf{h}_k$ and $\mathbf{y}_k$)
- for prediction of next time point in signals through any dimension (time), the FC net uses information in form the vector $\mathbf{h}$ from previous time point



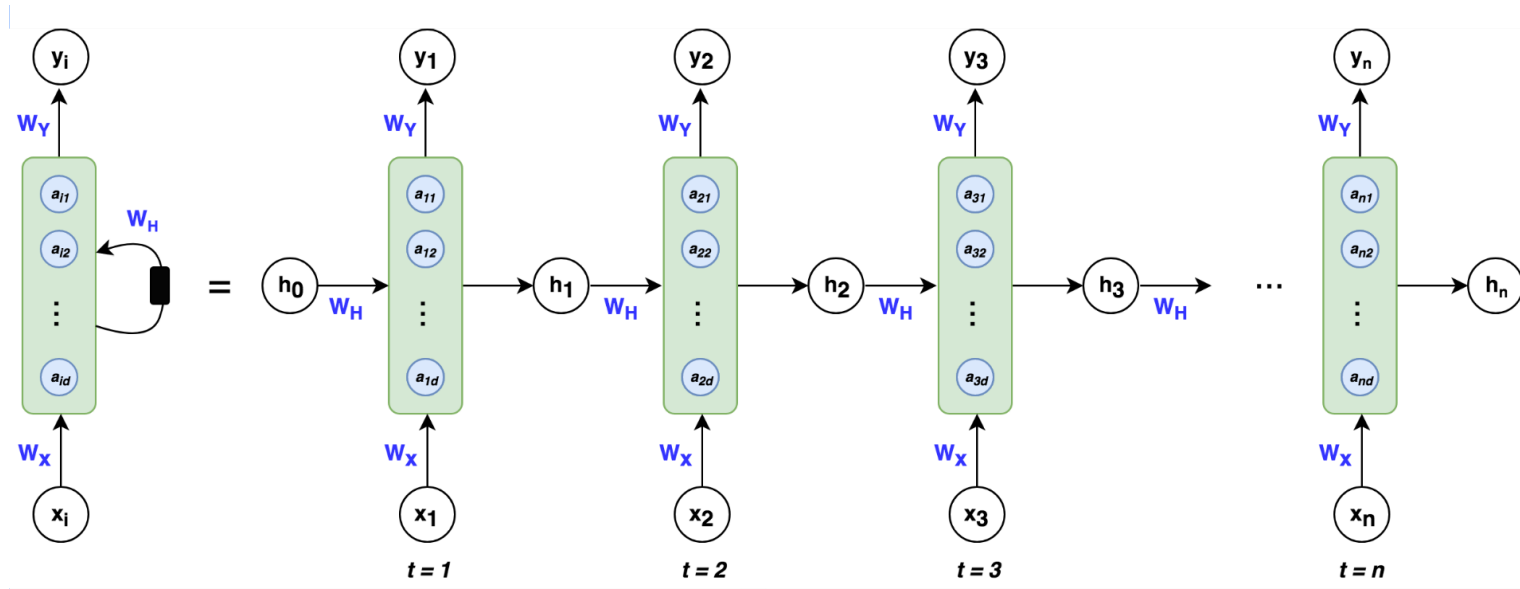
Architecture of Recurrent Networks

- It can be understood as series of FC layers sharing weights
- There are two inputs:
 - $\mathbf{x}_i$ is an input vector of data
 - $\mathbf{h}_{i-1}$ is a hidden layer vector of previous layer
- There are two outputs:
 - $\mathbf{y}_i$ is a vector of predicted values
 - $\mathbf{h}_i$ is a hidden layer vector
- There are three matrix of weights:
 - $\mathbf{W}_x$... weight matrix for transformation of input data data
 - $\mathbf{W}_y$... weight matrix for transformation of vector $\mathbf{h}_i$ to output $\mathbf{y}_i$
 - $\mathbf{W}_h$... weight matrix for transformation of vector $\mathbf{h}_{i-1}$ to output $\mathbf{h}_i$



Architecture of Recurrent Networks

Because the all weight matrices of FC for each “time” are shared, we can use **RECURSIVE APPROACH**



Vanilla recurrent network “Elman’s network”

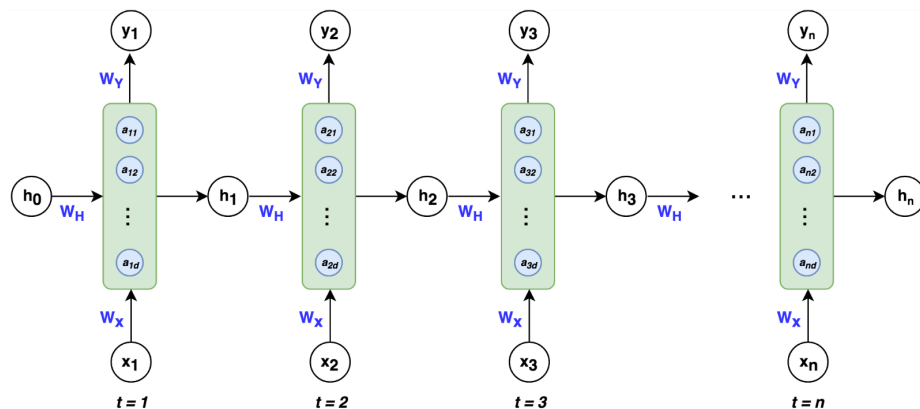
- The value of **hidden layer vector** $\mathbf{h}_t$ can be compute as follows:

$$\mathbf{h}_t = \alpha_h(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\mathbf{y}_t = \alpha_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$

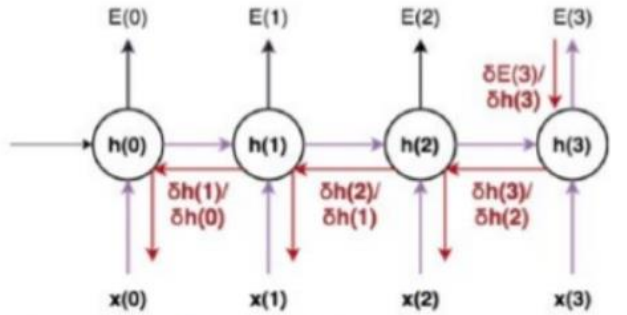
α_h ... activation function for calculation h vector
 α_y ... activation function for calculation y vector

- The networks share weight matrices
- Output is vector $\mathbf{y} = [y_1, \dots, y_n]$ of n predicted values
- Backprop. is based on an average or sum of error over “time”
- Information from older pattern is cumulated, but if we use sigmoid => gradual forgetting

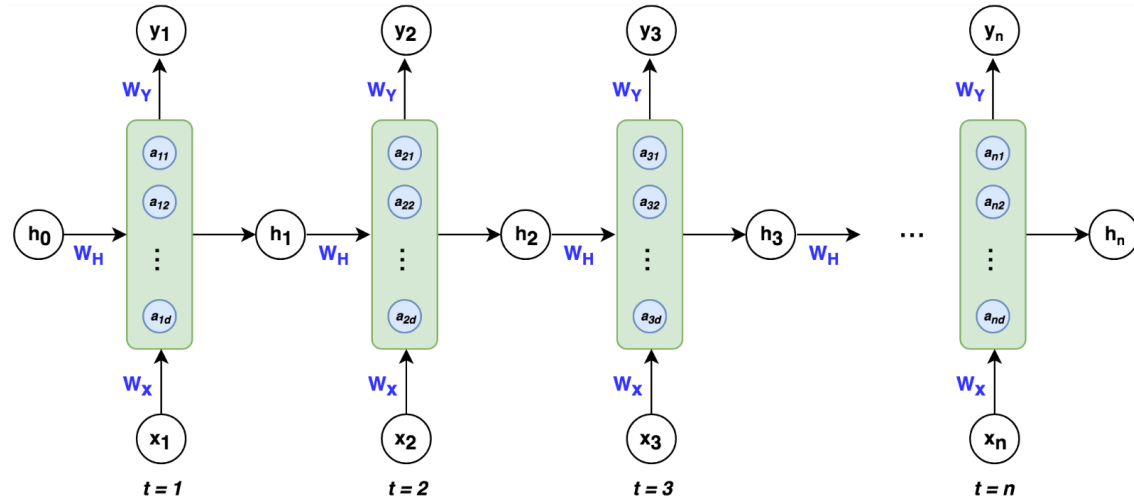


Pipeline

- 1) Set an initial values of weight matrices ($\mathbf{W}_x$, $\mathbf{W}_h$, $\mathbf{W}_y$)
- 2) Predict an output value for each time point $\mathbf{y} = [y_1, \dots, y_n]$
- 3) Compute an average error of prediction
- 4) Modified a weight matrices via backprop.
- 5) Repeat step 2-4



Example back-prop in time with 3 time-steps



Utilizing

- **Natural language processing**
 - obtaining the context from sentence/text
 - classification of text document
 - automatic translation
 - spam detection
- **Speech recognition**
 - transfer spoken word to written text
- **Financial prediction, ...**
- **Analysis of sequences:**
 - gene sequences (e.g. secondary structure prediction, ...)
 - biomedical signals (e.g. prediction of pathological stages, ...)
 - biomedical images (e.g. explain Images, video, ...)

...

Summary

Pros

- predictor, which uses older prediction in the form activation values (h_{i-1}) from previous “iteration”
- learned gradual forgetting older prediction
- information transfer over time (or another dimension)
- we can process sequence of different lengths
- if the input size is larger, the model size does not increase

Cons

- difficult training (depend on length of signals)
- for each “time” point, the prediction of networks is required – must be performed iteratively
- gradient vanishing and exploding problems
- cannot be parallelized
- cannot be used transfer learning

There are many modifications of RNNs (e.g., network doesn't use $\mathbf{W}_y$ or bi-directional, ...)

LSTM and GRU

Long short-term memory, Gated recurrent units

Principle of LSTM

- This network introduces into RNN the recurrent “forget gates”, which allow use very long memories.
- The LSTM approach solves vanishing gradient or exploding problem in backprop.
- It can work even given long delay between significant events

$$\mathbf{F}_t = \alpha_f (\mathbf{W}_f [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

$$\mathbf{I}_t = \alpha_i (\mathbf{W}_i [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

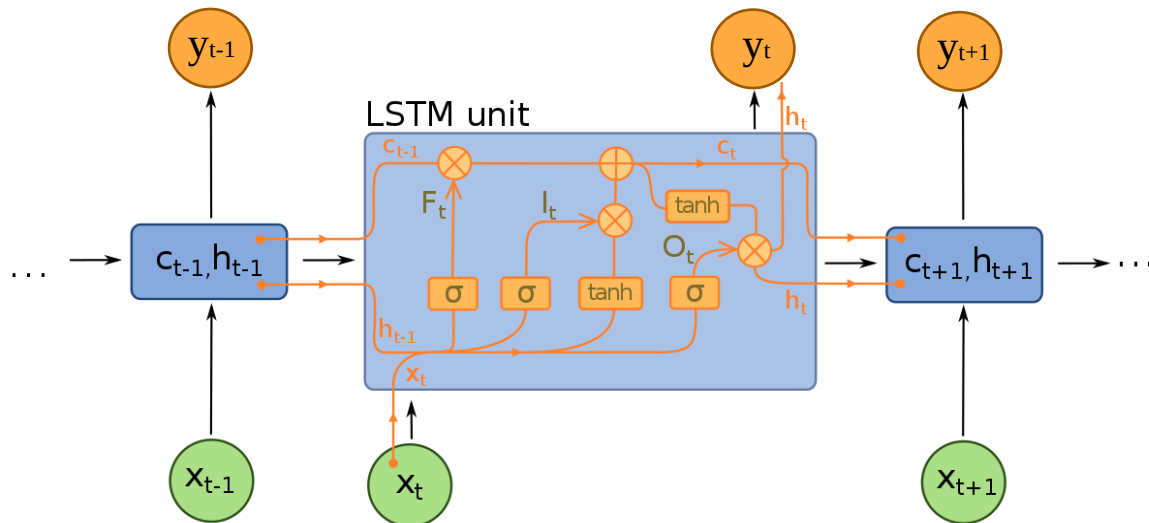
$$\tilde{\mathbf{C}}_t = \alpha_c (\mathbf{W}_c [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

$$\mathbf{C}_t = \mathbf{F}_t \mathbf{C}_{t-1} + \mathbf{I}_t \tilde{\mathbf{C}}_t$$

$$\mathbf{O}_t = \alpha_o (\mathbf{W}_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{O}_t \alpha_h (\mathbf{C}_t)$$

$$\mathbf{y}_t = \mathbf{h}_t$$



Summary

Pros

- very successful modification of RNNs and their utilizing
- solves a vanishing gradient problem (error can be backpropagated through time and layers)
- long memory
- we can process sequence of different lengths

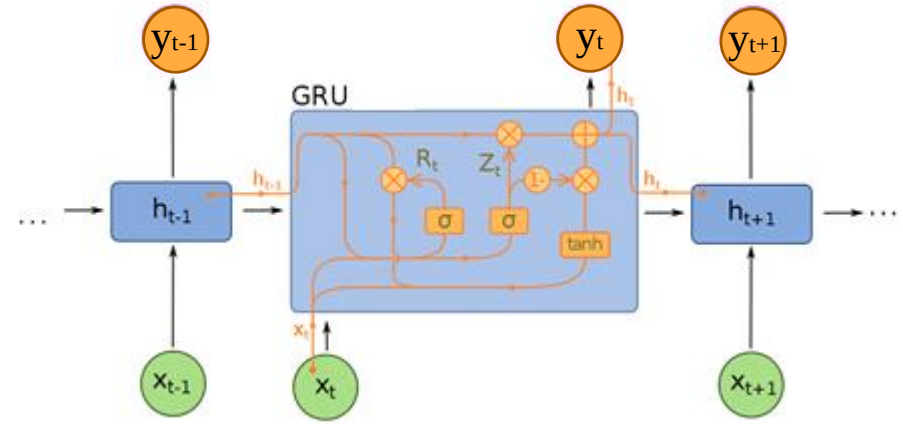
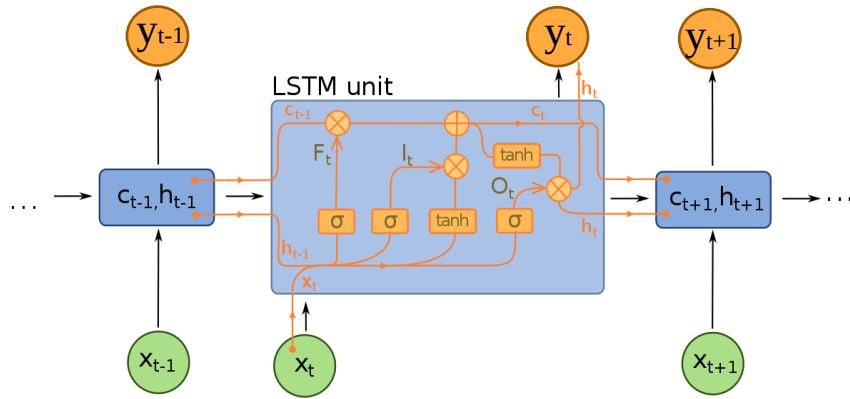
Cons

- uses sigmoid or tanh activation function – more prone to saturation (you can use “softsign”)
- Iterative calculation – high computational complexity
- cannot be parallelized
- cannot be used transfer learning
- cannot be re-trained – generalization problem

There are many modifications of the LSTM (e.g. adding “peephole connection”, Depth Gated RNNs, Clockwork RNNs, grid LSTM or **GRU**)

Principle of GRU

- This network introduces Gated Recurrent Units (GRU)
- It is simpler than LSTM and has fewer parameters
- In some cases, it has similar performance like LSTM



Modified RNN

with Attention modules

RNN with attention modules

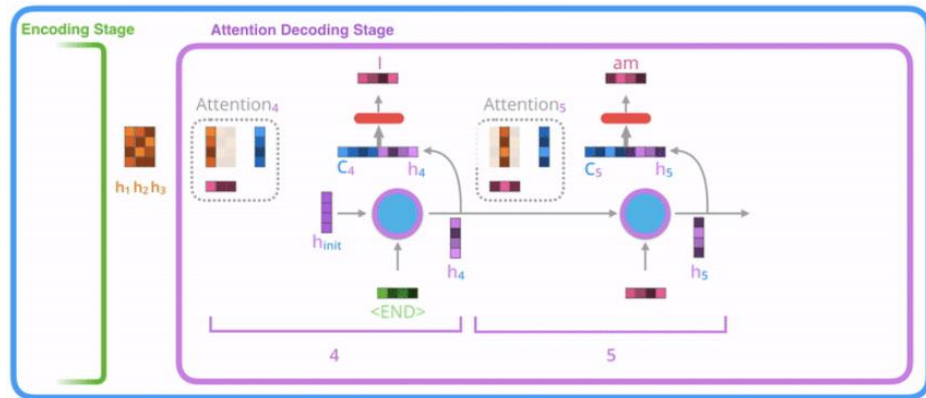
- Based on the fully connected networks
- Weighting of memory vectors – h
- Extract a relevant information in history
- Allows a global view and context information

Pros

- solves the vanishing and exploding gradient
- provides spatial information for image analysis
- focusing on relevant information with context
- uses all memory vectors in Decoding stage

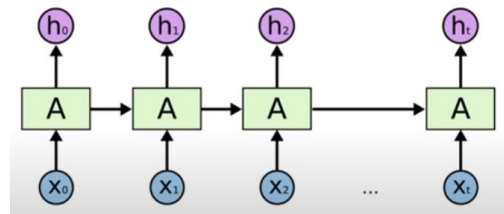
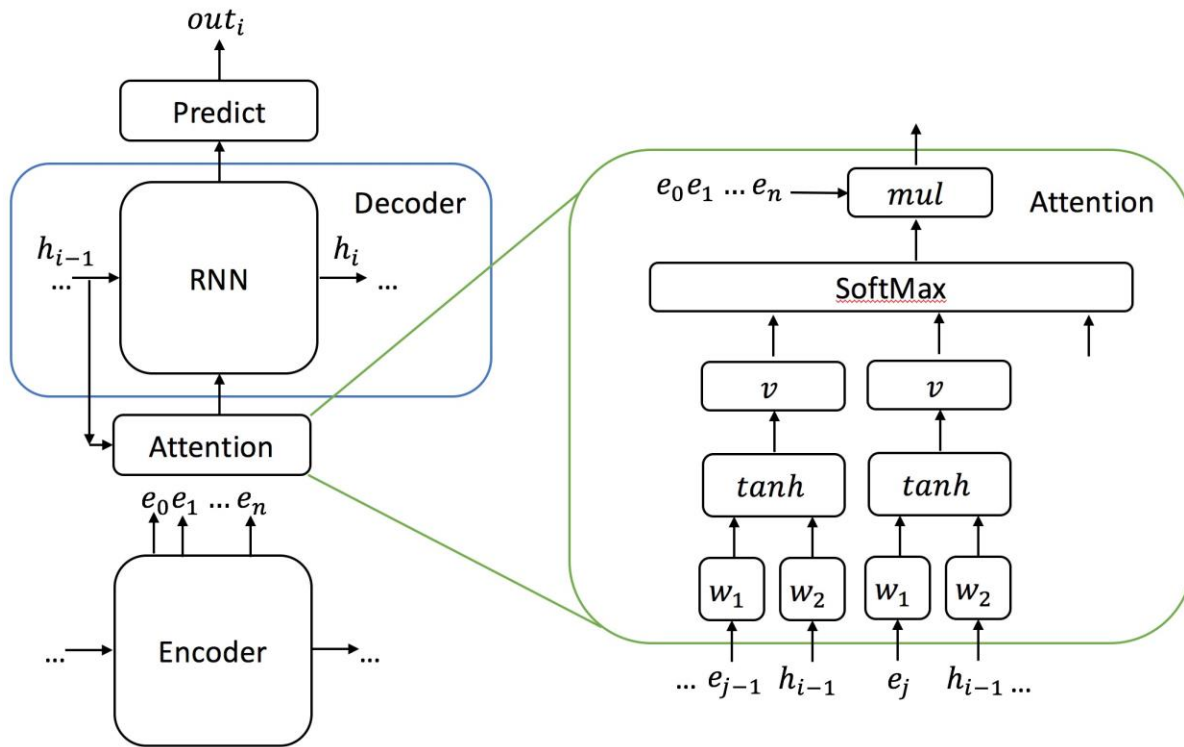
Cons

- high computational complexity (fully connected networks)
- minimal parallelization
- cannot be used a transfer learning

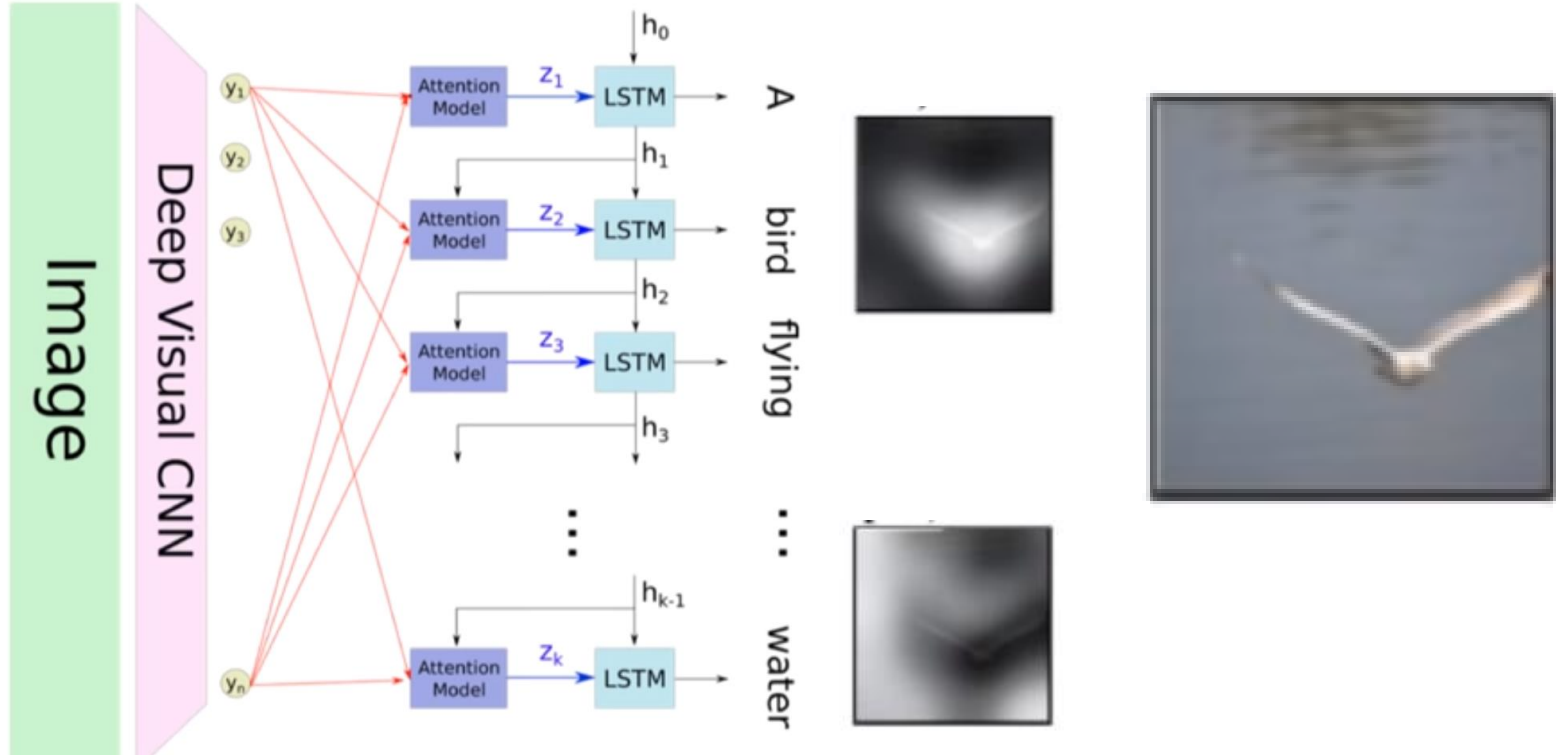


RNN with attention modules

Attention modules



Understanding of image



Transformer networks

Special fully connected networks

“The LSTM is dead. Long live the Transformers.”

Transformer networks

- based on the fully connected networks
- without CNN or RNN blocks
- alternative to RNN – different basics and architecture, but similar effect
- uses residual blocks and ReLU activation
- contains six encoders and six decoders
- each decoder has access to inputs from all encoders (features, hidden st

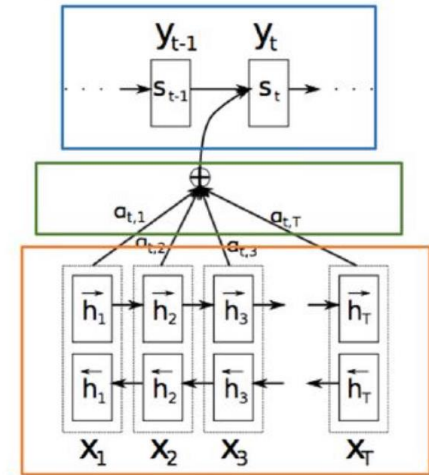
Pros

- easy training, high accuracy
- can be parallelized
- can be used pretrained nets and are re-trainable

Cons

- Problem with long signals (must be finite)
- Real-time is not possible

Model Architecture



Utilizing

Natural language processing

- Google Translator (2019) – hybrid model (RNN for decoder)

Object detection

- „End-to-end with Transformer“, 2020 (Facebook)

Image segmentation

- „TeTrIS: Template Transformer Networks for Image Segmentation With Shape Priors“, 2019

Restoration

- „Spatio-temporal Transformer Network for Video Restoration“, 2019

Image registration

- „Image-and-Spatial Transformer Networks for Structure-Guided Image Registration“, 2019

Modifications: „Dense TN“, „Kernel TN“, „Diffeomorphic TN“, „Graph TN“, „Template TN“